

# 敏感字段加密

## 题目描述

给定一个由多个命令字组成的命令字符串：

- 1. 字符串长度小于等于127字节，只包含大小写字母，数字，下划线和偶数个双引号；
- 2. 命令字之间以一个或多个下划线\_ 进行分割；
- 3. 可以通过两个双引号 "" 来标识包含下划线\_ 的命令字或空命令字（仅包含两个双引号的命令字），双引号不会在命令字内部出现；

请对指定索引的敏感字段进行加密，替换为\*\*\*\*\*（6个\*），并 删除命令 字前后多余的下划线\_。

如果无法找到指定索引的命令字， 输出字符串 ERROR。

## 输入描述

输入为两行，第一行为命令字索引 K（从 0 开始），第二行为命令字符串 S。

## 输出描述

输出处理后的命令字符串，如果无法找到指定索引的命令字，输出字符串ERROR

## 用例

|    |                                      |
|----|--------------------------------------|
| 输入 | 1<br>password__a12345678_timeout_100 |
| 输出 | password_*****_timeout_100           |
| 说明 | 无                                    |

|    |                                                |
|----|------------------------------------------------|
| 输入 | 2<br>aaa_password_"a12_45678"_timeout__100_""_ |
| 输出 | aaa_password_*****_timeout_100_""              |
| 说明 | 无                                              |

题目解析

本题只要考虑清楚一个问题就行：

什么样的 '\_' 是命令字 分隔符 ？

根据题意我们可以知道：

不在双引号内的 '\_' 是命令字的分隔符

因此，我们遍历输入串s的每个字符c时：

- 若 `c == '_'`，且 `c` 不在双引号范围内，则此时 `c` 是命令字分隔符，可以截断命令字。
- 其余情况，`c` 都可以当成命令字的组成字符。

那么如何判断 '\_' 是否在双引号范围内呢？

我们可以定义一个哨兵变量 `isQuotaOpen`,

- 若 `isQuotaOpen == true`，则说明当前双引号处于开启状态，后面遇到的字符都是双引号内部的。
- 若 `isQuotaOpen == false`，则说明当前双引号处于闭合状态，后面遇到的字符不在双引号内部

在我们遍历输入串 `s` 的过程中，每当遇到双引号字符，则对 `isQuotaOpen` 取反即可。

## JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6    const k = parseInt(await readline());
7    const s = await readline();
8
9    console.log(solution(s, k));
10 })();
11
12 function solution(s, k) {
13   const commands = [];
14
15   // 双引号是否处于开启状态
16   let isQuataOpen = false;
17
18   // 避免收尾操作，s 尾部追加一个 '_' 不影响结果
19   s += "_";
20
21   let command = [];
22
23   for (let i = 0; i < s.length; i++) {
24     const c = s[i];
25
26     // 如果遇到双引号，则先将 isQuataOpen 取反
27     if (c == '"') {
28       isQuataOpen = !isQuataOpen;
29     }
30
31     if (c != "_" || isQuataOpen) {
32       // 如果遇到的 c 不是 '_' 或者是双引号里面的 '_'，则此时 c 不是命令字分隔符，而是命令字一部分
```

运行

```

33     command.push(c);
34     } else if (command.length > 0) {
35         // 如果 c 是命令字分隔符, 且 command 命令字不为空, 则命令字被截断
36         commands.push(command.join(""));
37         // 清空command, 用于下个命令字的收集
38         command.length = 0;
39     }
40 }
41
42 if (k >= commands.length) {
43     // 如果无法找到指定索引的命令字, 输出字符串ERROR
44     return "ERROR";
45 } else {
46     // 对指定索引的敏感字段进行加密, 替换为***** (6个*), 并删除命令字前后多余的下划线_
47     commands[k] = "*****";
48     return commands.join("_");
49 }
50 }

```

## Java算法源码

```

1  import java.util.ArrayList;
2  import java.util.Scanner;
3  import java.util.StringJoiner;
4
5  public class Main {
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8
9          int k = sc.nextInt();
10         String s = sc.next();
11
12         System.out.println(solution(s, k));
13     }

```

```

14
15     public static String solution(String s, int k) {
16         ArrayList<String> commands = new ArrayList<>();
17
18         // 双引号是否处于开启状态
19         boolean isQuotaOpen = false;
20
21         // 避免收尾操作, s 尾部追加一个 '_' 不影响结果
22         s += "_";
23
24         StringBuilder command = new StringBuilder();
25         for (int i = 0; i < s.length(); i++) {
26             char c = s.charAt(i);
27
28             // 如果遇到双引号, 则先将 isQuotaOpen 取反
29             if (c == '"') {
30                 isQuotaOpen = !isQuotaOpen;
31             }
32
33             if (c != '_' || isQuotaOpen) {
34                 // 如果遇到的 c 不是 '_' 或者是双引号里面的 '_', 则此时 c 不是命令字分隔符, 而是命令字一部分
35                 command.append(c);
36             } else if (command.length() > 0) {
37                 // 如果 c 是命令字分隔符, 且 command 命令字不为空, 则命令字被截断
38                 commands.add(command.toString());
39                 // 清空command, 用于下个命令字的收集
40                 command = new StringBuilder();
41             }
42         }
43
44         if (k >= commands.size()) {
45             // 如果无法找到指定索引的命令字, 输出字符串ERROR
46             return "ERROR";
47         } else {
48             // 对指定索引的敏感字段进行加密, 替换为***** (6个*), 并删除命令字前后多余的下划线_
49             commands.set(k, "*****");
50             return String.join("_", commands);

```

```
51 |         }52 |     }
53 | }
```

## Python算法源码

```
1  # 输入获取
2  k = int(input())
3  s = input()
4
5
6  # 算法入口
7  def solution():
8      global s
9
10     commands = []
11
12     # 双引号是否处于开启状态
13     isQuotaOpen = False
14
15     # 避免收尾操作, s 尾部追加一个 '_' 不影响结果
16     s += "_"
17
18     command = []
19     for c in s:
20         # 如果遇到双引号, 则先将 isQuotaOpen 取反
21         if c == '"':
22             isQuotaOpen = not isQuotaOpen
23
24         if c != '_' or isQuotaOpen:
25             # 如果遇到的 c 不是 '_' 或者是双引号里面的 '_', 则此时 c 不是命令字分隔符, 而是命令字一部分
26             command.append(c)
27         elif len(command) > 0:
28             # 如果 c 是命令字分隔符, 且 command 命令字不为空, 则命令字被截断
29             commands.append("".join(command))
```

```

30         # 清空command, 用于下个命令字的收集31 |         command.clear()
32
33     if k >= len(commands):
34         # 如果无法找到指定索引的命令字, 输出字符串ERROR
35         return "ERROR"
36     else:
37         # 对指定索引的敏感字段进行加密, 替换为***** (6个*), 并删除命令字前后多余的下划线_
38         commands[k] = "*****"
39         return "_".join(commands)
40
41
42 # 算法调用
43 print(solution())

```

## C算法源码

```

1  #include <stdio.h>
2  #include <stdbool.h>
3  #include <string.h>
4
5  #define MAX_LEN 200
6  #define MAX_SIZE 200
7
8  int main() {
9      int k;
10     scanf("%d", &k);
11
12     char s[MAX_LEN] = {'\0'};
13     scanf("%s", s);
14
15     // 记录所有命令字
16     char commands[MAX_SIZE][MAX_LEN] = {'\0'};
17     int commands_size = 0;
18
19     // 双引号是否处于开启状态

```

```

20     bool isQuotaOpen = false;
21
22     // 避免收尾操作, s 尾部追加一个 '_' 不影响结果
23     s[strlen(s)] = '_';
24
25     // 临时记录单个命令字
26     char command[MAX_LEN] = {'\0'};
27     int command_size = 0;
28
29     for (int i = 0; i < strlen(s); i++) {
30         char c = s[i];
31
32         // 如果遇到双引号, 则先将 isQuotaOpen 取反
33         if (c == '"') {
34             isQuotaOpen = !isQuotaOpen;
35         }
36
37         if (c != '_' || isQuotaOpen) {
38             // 如果遇到的 c 不是 '_' 或者是双引号里面的 '_', 则此时 c 不是命令字分隔符, 而是命令字一部分
39             command[command_size++] = c;
40         } else if (command_size > 0) {
41             // 如果 c 是命令字分隔符, 且 command 命令字不为空, 则命令字被截断
42             strcpy(commands[commands_size++], command);
43             // 清空command, 用于下个命令字的收集
44             memset(command, '\0', MAX_LEN);
45             command_size = 0;
46         }
47     }
48
49     if (k >= commands_size) {
50         // 如果无法找到指定索引的命令字, 输出字符串ERROR
51         puts("ERROR");
52     } else {
53         // 对指定索引的敏感字段进行加密, 替换为***** (6个*), 并删除命令字前后多余的下划线_
54         strcpy(commands[k], "*****");
55
56         for (int i = 0; i < commands_size; i++) {

```

```

57         printf("%s", commands[i]); 58
59         if (i < commands_size - 1) {
60             printf("_");
61         }
62     }
63 }
64
65 return 0;
66 }

```

## C++ 算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  string solution(string &s, int k) {
6      vector<string> commands;
7
8      // 双引号是否处于开启状态
9      bool isQuotaOpen = false;
10
11     // 避免收尾操作, s 尾部追加一个 '_' 不影响结果
12     s += "_";
13
14     string command;
15     for (const auto &c: s) {
16         // 如果遇到双引号, 则先将 isQuotaOpen 取反
17         if (c == '"') {
18             isQuotaOpen = !isQuotaOpen;
19         }
20
21         if (c != '_' || isQuotaOpen) {
22             // 如果遇到的 c 不是 '_' 或者是双引号里面的 '_', 则此时 c 不是命令字分隔符, 而是命令字一部分

```

```

23         command += c;24         } else if (!command.empty()) {
25         // 如果 c 是命令字分隔符, 且 command 命令字不为空, 则命令字被截断
26         commands.emplace_back(command);
27         // 清空command, 用于下个命令字的收集
28         command = "";
29     }
30 }
31
32 if (k >= commands.size()) {
33     // 如果无法找到指定索引的命令字, 输出字符串ERROR
34     return "ERROR";
35 } else {
36     // 对指定索引的敏感字段进行加密, 替换为***** (6个*), 并删除命令字前后多余的下划线_
37     commands[k] = "*****";
38
39     string res;
40     for (int i = 0; i < commands.size(); i++) {
41         res += commands[i];
42
43         if (i < commands.size() - 1) {
44             res += "_";
45         }
46     }
47
48     return res;
49 }
50 }
51
52 int main() {
53     int k;
54     cin >> k;
55
56     string s;
57     cin >> s;
58
59     cout << solution(s, k) << endl;
60

```

```
61 |     return 0;  
62 | }
```